

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 1.2]

MODELLING CONCURRENT PROGRAM EXECUTION

SUMMARY

- Making models of concurrent programs
 - State diagrams & execution scenarios
- Correctness
 - safety and liveness properties
 - fairness
- Critical Section Problem
 - path towards Dekker's algorithm
- Verification of Concurrent Programs
 - linear temporal logic - LTL

FROM PROGRAMS TO MODELS (AND BACK)

- Importance of models and abstraction for computer science and engineering in particular
 - descrizione / rappresentazione rigorosa della **model: rigorous description / representation of program (system)** ad un adeguato **structure and behaviour at a proper level of abstraction**
 - including relevant information, abstracting from non-relevant aspects
 - diagrammatical representations for program design
 - formal models for program analysis and verification
- **Defining proper models for concurrent programs**
 - defining models for the structure and behaviour of concurrent programs abstracting from the low-level details of their actual implementation and realisation
 - progettazione **design**
 - enabling the possibility to reason about their dynamic behaviour of concurrent programs
 - **verification** (Verificare che il programma sia corretto o meno)

A MODEL FOR CONCURRENT PROGRAM EXECUTION

- Modelling each process as a sequence of **atomic actions**, each action corresponding to the atomic execution of a statement istruzione
- Speed independence assumption** => L'ipotesi di indipendenza dalla velocità presuppone che le velocità relative dei processi siano sconosciute e potenzialmente variabili, purché siano positive (ovvero, i processi procedano).

modelling the execution of a concurrent program as a sequence of actions ottenuta alternando arbitrariamente obtained by **arbitrarily interleaving** the actions (atomic istruzioni statements) from the processes Poiché non vengono formulate ipotesi sulla velocità di esecuzione dei processi, è sicuro modellare i programmi concorrenti alternando arbitrariamente le azioni (istruzioni atomiche) di ciascun processo.

- a single abstract *global processor* executing all the actions
- **atomic** statements fino al => executed to completion without the possibility of interleaving (devo eseguire fino alla fine, quando inizio, tutto lo statement atomico, senza interleaving di altri processi: Un'azione è atomica se, una volta iniziata, non può essere interrotta.)
- during the computation the *control pointer* or instruction of a process indicates the next statement that can be executed by that process
- a **computation** or **scenario** is an execution sequence verificarsi that can occur come as a result of the interleaving 

Uno scenario è una sequenza di esecuzione generata dall'interleaving delle azioni. Poiché la velocità dei singoli processi è sconosciuta (assunzione di speed independence), la "storia" effettiva di ciò che accade nel sistema rappresenta solo una delle tantissime combinazioni possibili. Dato che il processore può scegliere arbitrariamente quale azione eseguire tra quelle pronte nei vari processi, l'esecuzione di un programma concorrente non segue un percorso lineare, ma si sviluppa come un albero di possibilità. In questo modello, ogni ramo dell'albero rappresenta uno specifico scenario sequenziale, mentre ogni nodo fotografa lo stato della memoria e la posizione dei puntatori d'istruzione in quel preciso istante. Di conseguenza, quando si progetta un sistema concorrente, è fondamentale garantire che tutti i possibili scenari portino a un risultato corretto. Se esiste anche una sola sequenza su un milione in cui l'interleaving causa un comportamento anomalo (come una race condition, in cui due processi scrivono contemporaneamente sulla stessa variabile), il programma contiene un bug di concorrenza.

FIRST TRIVIAL EXAMPLE

Due processi che hanno memoria condivisa e locale

integer n := 0	
p	q
integer k1 := 1	integer k2 := 2
p1: n := k1	q1: n := k2

- Each labeled line represents an atomic statement
- Each process has private memory
 - local variables, such as k1 and k2
- Processes shares some memory
 - global variables, such as n
- Program execution: 2 scenarios
 - p1, q1 ($\Rightarrow$ n finally is equal to 2) prima p1, poi q1
 - q1, p1 ($\Rightarrow$ n finally is equal to 1) prima q1, poi p1

STATE DIAGRAMS

modello ad interleaving basato su Processi come sequenze di azioni atomiche, Assunzione Speed Independence e il Processore Globale Astratto

- Given the model, the execution of a concurrent program can be formally represented by **states** and **transitions** between states
 - the state is defined by a tuple consisting of
 - one element of each process that is a label (statement) from that process
 - one element for each global or local variable that is a value whose type is the same as the type of a variable
 - there is a transition between two states s_1 and s_2 if executing a statement in state s_1 changes the state to s_2
 - the statement executed must be one of those pointed  by a control pointer in s_1
- The **state diagram** is a *graph* containing all the *reachable states* of the programs
 - scenarios are represented by ~~directed paths~~ ^{percorsi orientati} through the state diagram from the initial state
 - cycles represent the possibility of infinite computation in a finite graph

FIRST EXAMPLE: STATE DIAGRAM

- State tuple: $\langle p, q, n, k1, k2 \rangle$

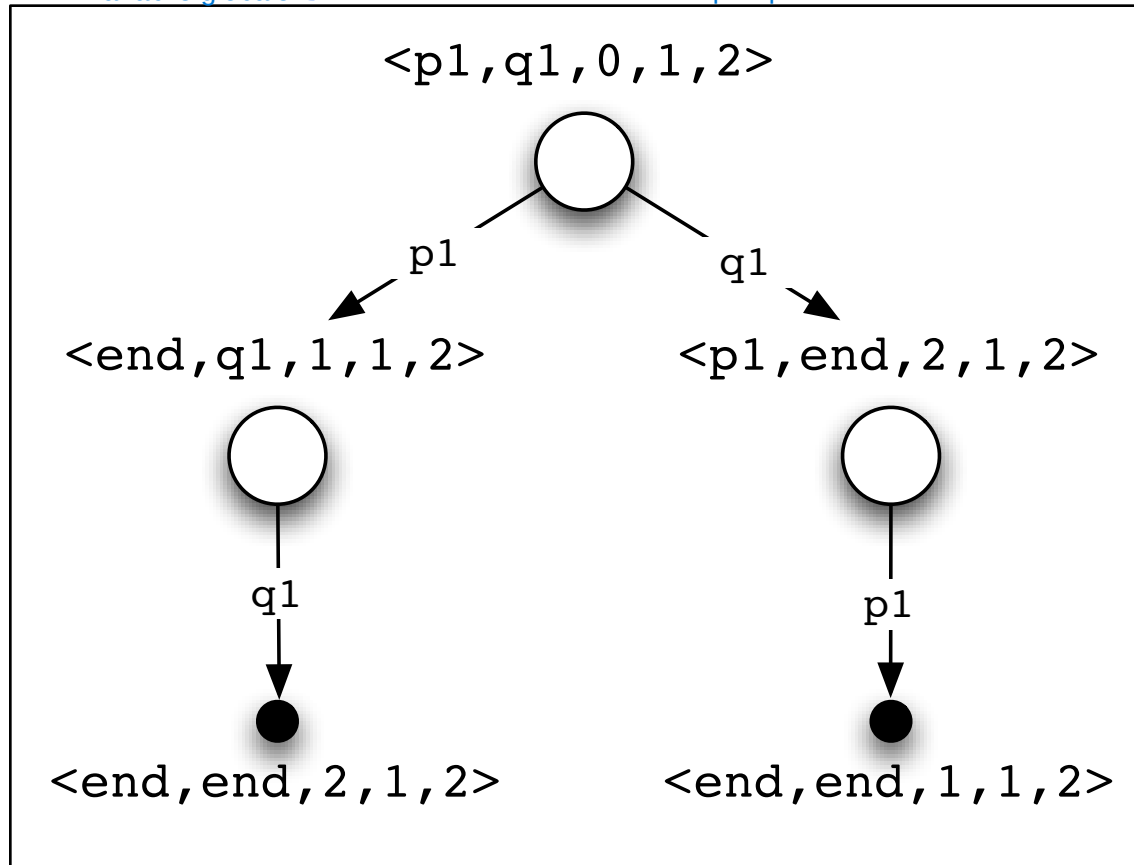
a che istruzione sono
nel processo p?

a che istruzione sono
nel processo q?

variabile globale

variabili locali di p e q

Un Labeled Transition System (LTS) è proprio il formalismo matematico usato per descrivere quello che abbiamo chiamato "diagramma di stato" di un programma concorrente.



Transizioni sono etichettate dalle istruzioni che scelgo di eseguire atomicamente

- 5 states, 2 scenarios

Nella pratica, non tutti gli scenari sono equiprobabili

EXAMPLE #2

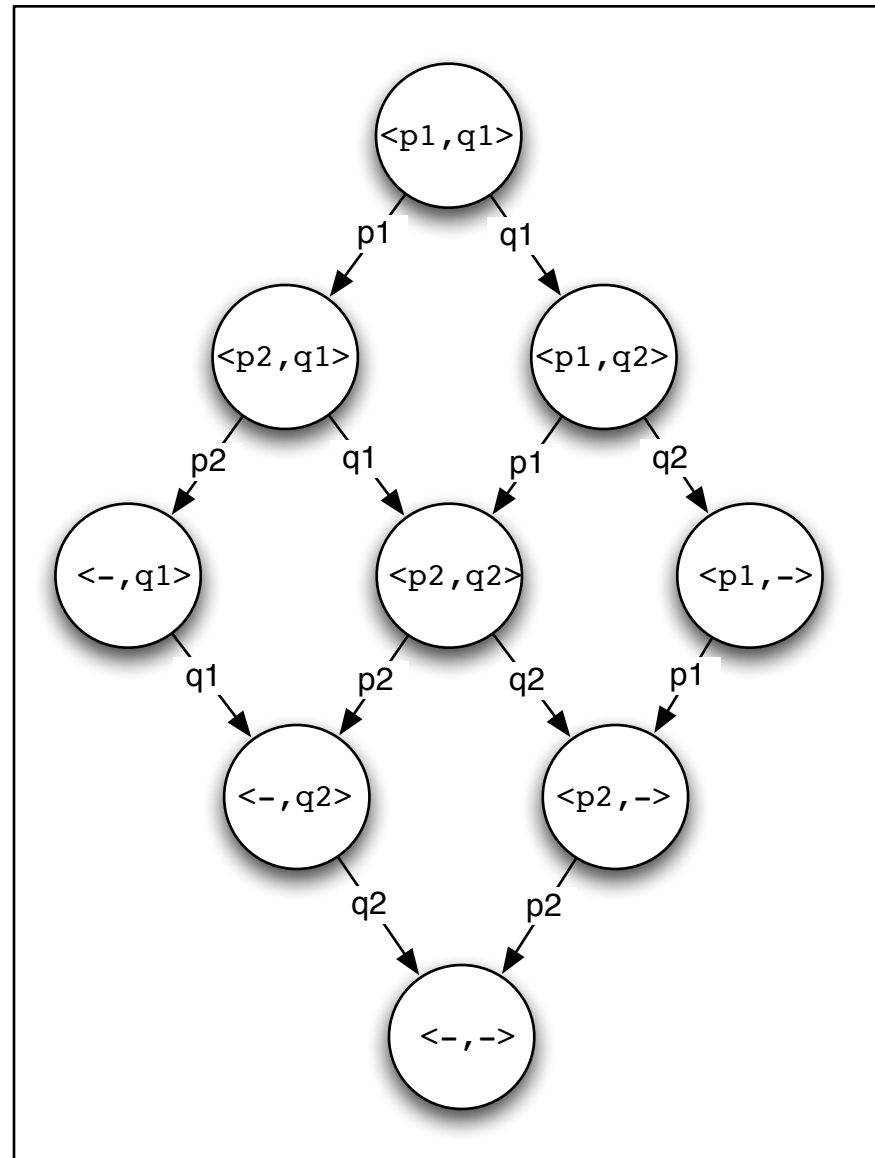
p	q
p1: print("p1") p2: print("p2")	q1: print("q1") q2: print("q2")

- State diagram?
- How many scenarios?

EXAMPLE #2

- 6 scenarios:

p1	p2	q1	q2
p1	q1	p2	q2
p1	q1	q2	p2
q1	q2	p1	p2
q1	p1	q2	p2
q1	p1	p2	q2



EXAMPLE #3

p	q
p1: print("p1") p2: print("p2") p3: print("p3")	q1: print("q1") q2: print("q2") q3: print("q3")

- Scenarios?

EXAMPLE #3

- 20 scenarios:

p1	p2	p3	q1	q2	q3
p1	p2	q1	p3	q2	q3
p1	q1	p2	p3	q2	q3
q1	p1	p1	p3	q2	q3
p1	p2	q1	q2	p3	q3
p1	q1	p2	q2	p3	q3
q1	p1	p2	q2	p3	q3
p1	q1	q2	p2	p3	q3
q1	p1	q2	p2	p3	q3
q1	q2	p1	p2	p3	q3
p1	p2	q1	q2	q3	p3
p1	q1	p2	q2	q3	p3
q1	p1	p2	q2	q3	p3
p1	q1	q2	p2	q3	p3
q1	p1	q2	p2	q3	p3
q1	q2	p1	p2	q3	p3
p1	q1	q2	q3	p2	p3
q1	p1	q2	q3	p2	p3
q1	q2	p1	q3	p2	p3
q1	q2	q3	p1	p2	p3

NUM. SCENARIOS WITH 2 PROCESSES

num. actions per proc.	num. scenarios
1	2
2	6
3	20
4	70
5	252
6	924
7	3432
8	12820
...	

- Note: in these cases process actions have no dependencies...

EXAMPLE #4

- Changing the number of processes: 3 processes..

p	q	r
p1: print("p1") p2: print("p2")	q1: print("q1") q2: print("q2")	r1: print("r1") r2: print("r2")

- Scenarios?

EXAMPLE #4

- 90 scenarios

```
p1 p2 q1 q2 r1 r2
p1 p2 q1 r1 q2 r2
p1 p2 q1 r1 r2 q2
p1 p2 r1 q1 q2 r2
p1 p2 r1 q1 r2 q2
p1 p2 r1 r2 q1 q2
p1 q1 p2 q2 r1 r2
...
q1 p1 p2 q2 r1 r2
q1 p1 p2 r1 q2 r2
q1 p1 p2 r1 r2 q2
q1 p1 q2 p2 r1 r2
q1 p1 q2 r1 p2 r2
...
r1 p1 p2 q1 q2 r2
r1 p1 p2 q1 r2 q2
r1 p1 p2 r2 q1 q2
r1 p1 q1 p2 q2 r2
r1 p1 q1 p2 r2 q2
...
r1 r2 q1 p1 q2 p2
r1 r2 q1 q2 p1 p2
```

NUM. SCENARIOS WITH 3 PROCESSES

num. actions per proc.	num. scenarios
1	6
2	90
3	1680
4	34650
...	...

GENERALIZING...

La formula serve a calcolare esattamente quanti sono i possibili scenari (le diverse sequenze di esecuzione) che possono generarsi mescolando le istruzioni di più processi.

- Number of scenarios produced by n processes, each having m_i actions:

questa è la formula delle permutazioni con ripetizione.

$$ns = \frac{(\sum_{i=1}^n m_i)!}{\prod_{i=1}^n (m_i!)}$$

numero di processi totali

numero di azioni atomiche nel processo i-esimo.

representa le permutazioni totali di tutte le istruzioni come se fossero tutte distinte.

serve a eliminare le combinazioni impossibili, perché all'interno di ogni singolo processo l'ordine delle istruzioni è fisso (l'azione 1 deve sempre avvenire prima dell'azione 2 dello stesso processo).

Esplosione degli Stati

	$n = 2$	3	4	5	6
$m_i = 2$	6	90	2520	113400	$2^{22.8}$
3	20	1680	$2^{18.4}$	$2^{27.3}$	$2^{36.9}$
4	70	34650	$2^{25.9}$	$2^{38.1}$	$2^{51.5}$
5	252	$2^{19.5}$	$2^{33.4}$	$2^{49.1}$	$2^{66.2}$
6	924	$2^{24.0}$	$2^{41.0}$	$2^{60.2}$	$2^{81.1}$

“THE IMPORTANCE OF BEING ATOMIC”

Stesso esempio, ma due tipi di atomicità a livello di istruzione

• Atomic increment (1)

integer n := 0	
p	q
p1: n := n + 1	q1: n := n + 1

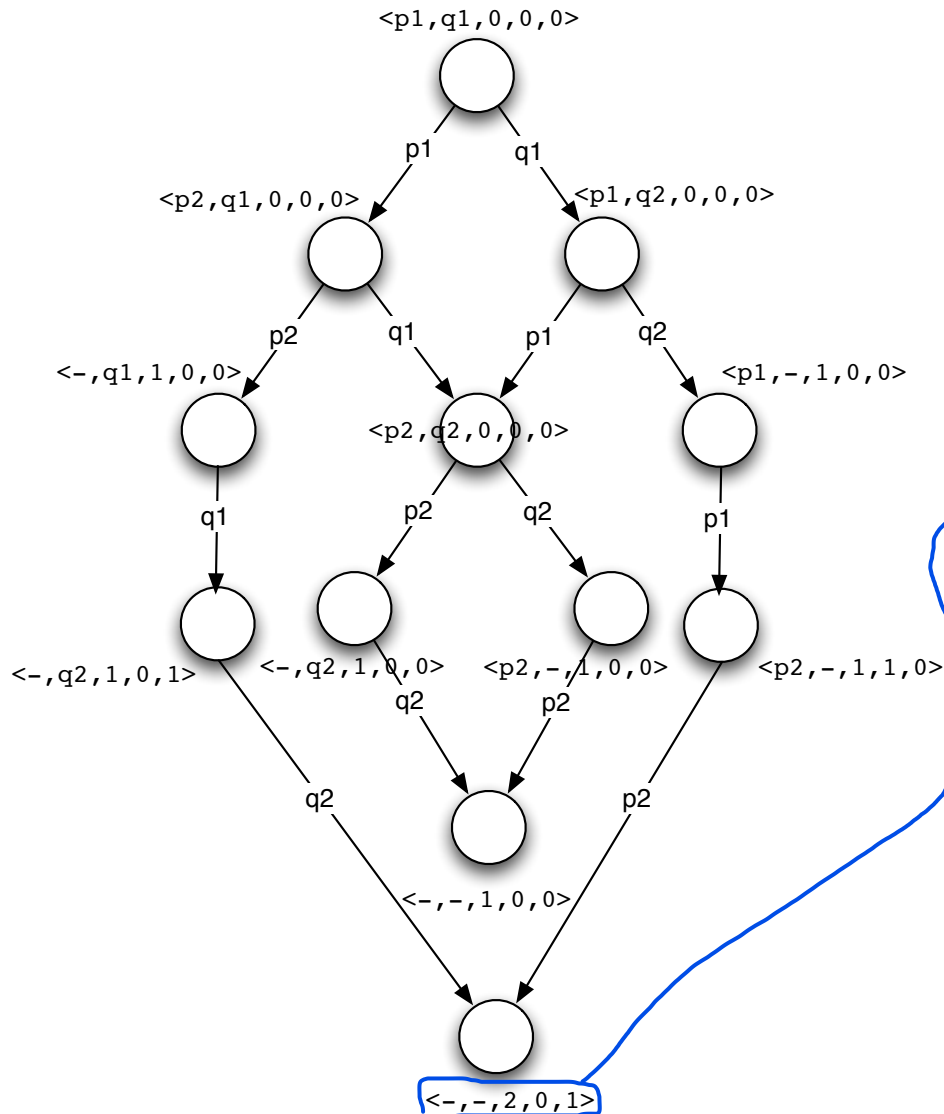
• Non-atomic increment (2)

integer n := 0	
p	q
integer tmp; p1: tmp := n p2: n := tmp + 1	integer tmp; q1: tmp := n q2: n := tmp + 1

sono due var. tmp diversi

- In the second case, scenarios exist in which the final value of n is 1 ad esempio, p1 q1 p2 q2 (perchè in tmp ho 0, entrambi metto n=1 ed il risultato è 1: in realtà, dovrebbe essere = 2 come sopra)

NON ATOMIC CASE: STATE DIAGRAM



12 states

Only 2 scenarios over 6
have final n value = 2

Un unico stato dove due scenari arrivano a quello stato

Quindi, si costruisce il grafo
e poi capire se valgono le
proprietà di correttezza

ASSIGNMENTS & INCREMENTS AT THE MACHINE-CODE LEVEL

- Stack machines

integer n := 0	
p	q
p1: push n p2: push #1 p3: add p4: pop n	q1: push n q2: push #1 q3: add q4: pop n

- Register machines

integer n := 0	
p	q
p1: load R1, n p2: add R1, #n p3: store n, R1	q1: load R1, n q2: add R1, #n q3: store n, R1

NON-ATOMIC STRUCTURES (1/3)

- The notion of “atomic” ^{può riferirsi} ~~can be referred~~ not only to actions, but also to data structures:
 - a data object is defined *atomic* ^{può trovarsi} ~~if it can be~~ in a finite number of states equals to the number of values that it can assume
 - operations change (atomically) that state
 - typically primitive data type in concurrent languages are atomic
 - not always: e.g. `double` in Java
- Abstract data types composed by multiple simpler data objects are typically non atomic
 - es: class in OO languages, structs in C

Se una variabile può valere 0 o 1, e io la cambio, essa passa da 0 a 1 senza mai passare per stati intermedi.

Una sequenza di istruzioni può essere considerata atomica se, anche se ci può essere interleaving, osservandola/analizzandola, essa non porta a nessun tipo di interferenza o problema ad equipararlo ad un'istruzione atomica

NON-ATOMIC STRUCTURES (2/3)

- In that case for the ADT (or more generally data object) it is possible to identify two basic types of states: *internal* and *external*

- the internal state is ^{rilevante} ~~meaningful~~ for who defines the data object (class)

Durante un'operazione (es. un bonifico), l'oggetto deve modificare diverse variabili una alla volta. In quei micro-istanti, lo stato interno dell'oggetto cambia continuamente.

- the external state is ^{rilevante} ~~meaningful~~ for who uses the data object

Immagina un oggetto ContoBancario. Gli stati esterni validi sono quelli in cui il saldo riflette operazioni concluse (es. "Ho 100€" oppure "Ho 50€"). Chi usa la classe si aspetta che l'oggetto sia sempre in uno di questi stati coerenti.

- The correspondence among internal and external states is *partial*

Significa che esistono molti stati interni che non corrispondono a nessuno stato esterno valido. Questi stati sono definiti incoerenti (non consistent).

- there exist internal states which have no a correspondent external state
- internal states which have a correspondent external state are defined **consistent**

Uno stato interno è consistente solo se "ha un corrispondente stato esterno".
Ovvero: se un osservatore esterno guarda l'oggetto in quel momento, vede qualcosa che ha senso e che rispetta le regole del programma.

Il problema delle strutture non atomiche è che mettono a nudo i loro stati interni incoerenti.
Se l'oggetto fosse atomico, passerebbe da "Stato Esterno 1" a "Stato Esterno 2" in un colpo solo.
Essendo non atomico, l'oggetto "si apre", mostra le sue viscere (stati interni) e, se sfortunatamente un altro processo guarda proprio in quel momento, il sistema fallisce perché legge dati che non dovrebbero esistere.

NON-ATOMIC STRUCTURES (3/3)

- Then, the execution of an operation on a (not-atomic) ADT can go through states that are *not consistent* (es: oggetti standard non sono Thread-safe)
 - e.g. a simple list
 - This is not a problem in the case of sequential programming
 - thanks to information hiding
 - Conversely, it is a problem in the case of concurrent programming
 - it can happen that a process would work on an object while the object is in an inconsistent state, since a process is concurrently operating on it
- > it is necessary to introduce proper mechanisms that would guarantee that processes work on data objects that are always in states that are consistent
- (ADT Thread-safe: permette di eseguire su di esse delle istruzioni in modo atomico. Dire che un ADT è Thread-safe significa che la classe è stata progettata in modo tale da nascondere gli stati interni inconsistenti durante l'uso concorrente.)

CYCLIC PROCESSES

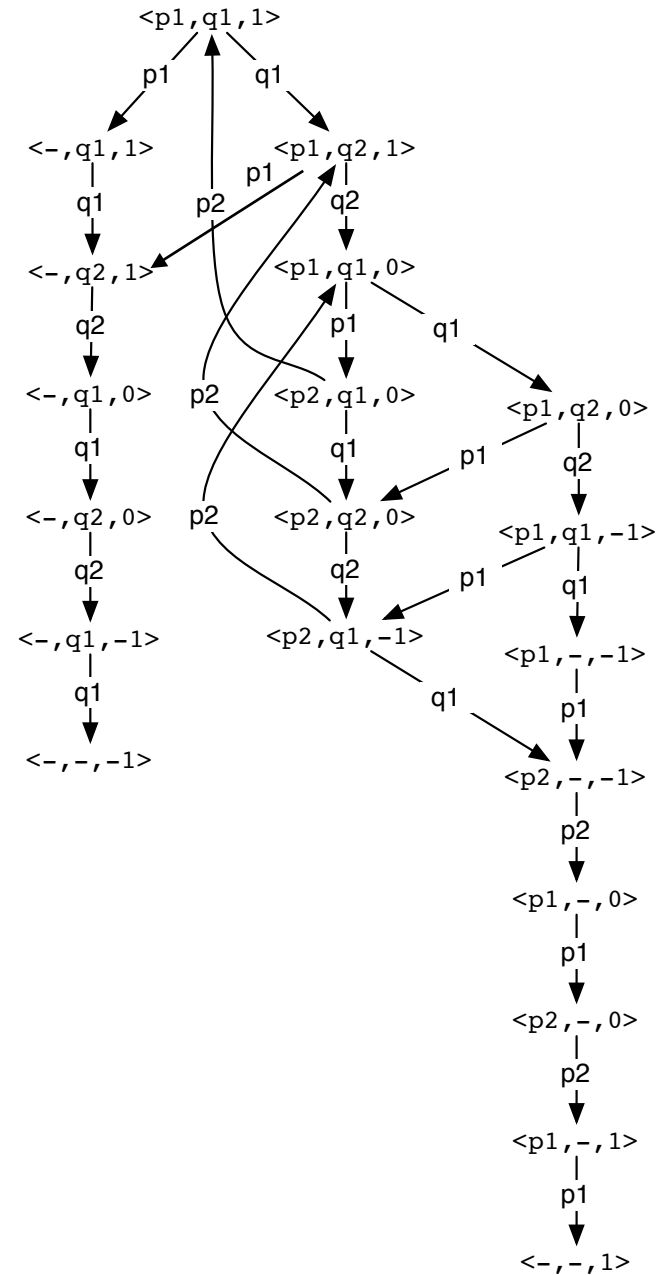
- **p** and **q** processes cycling on a condition

integer n := 1	
p	q
p1: while (n < 1) p2: n := n + 1	q1: while (n >= 0) q2: n := n - 1

Numero di stati finito
Numero di transizioni/istruzioni finito

- Analysis
 - state diagram ?
 - construct a scenario in which the loop in p executes exactly one
 - construct a scenario in which the loop in p executes exactly three times

STATE DIAGRAM



IS THIS MODEL A GOOD MODEL ?

OR RATHER: IS THE CONCURRENT PROGRAMMING ABSTRACTION JUSTIFIABLE ?

- Actually in the reality computer system **has not a global state**
 - matter of physics In un sistema distribuito, non c'è un "orologio globale" perfetto.
- That's ~~the~~ the role of abstraction: *we create a model of the system in which a kind of global entity executes the concurrent program by arbitrarily interleaving statements*
 - per facilitare l'analisi ~~to ease analysis~~
- But.... is it a valid model for real concurrent computing systems? $\Rightarrow$ *reality check* Questo modello interleaved è ancora valido in tutti e 4 i tipi di sistemi
 - multitasking systems C'è un solo processore fisico che salta da un processo all'altro (context switch). L'interleaving è esattamente ciò che accade nella realtà: il processore esegue un pezzetto di A, poi un pezzetto di B.
 - multicore systems
 - multiprocessor computers
 - distributed systems

ARBITRARILY INTERLEAVING: ABSTRACTING FROM TIME

Se il sistema è corretto per ogni interleaving, allora è corretto anche se una rete rallenta o se un processore accelera.

Ignorare il Tempo

- **Arbitrary interleaving** means that we ignore time in our analysis of concurrent programs
 - focussing only to
 - partial orders related to action sequences $a_1, a_2, \dots$
 - atomicity of the individual action $a_j \Rightarrow$ choosing what is atomic is fundamental
 - robustness with respect to both hardware (processor) and software changes
 - independent from changes in timings / performance
- This makes concurrent programs ~~amenable to formal analysis~~, which is necessary to ~~ensure correctness~~ of concurrent programs.
 - ~~proving correctness besides the actual execution time~~, which is typically strictly dependent on processors speed and system's environment timings

Nel mondo reale, un'azione A potrebbe durare 2 millisecondi e un'azione B 5 millisecondi. Verrebbe da pensare che A finisca sempre prima di B. L'interleaving arbitrario dice: "Ignora il tempo".

Sappiamo che nell'ambito del Processo A, a_1 deve venire prima di a_2 . Ma non sappiamo nulla della relazione tra a_1 e b_1 . L'analisi si concentra solo sul rispettare i vincoli interni di ogni processo, lasciando che il "mescolamento" con gli altri processi sia libero.

ideali all'analisi formale

garantire la

dimostrando la correttezza indipendentemente dal tempo di esecuzione effettivo

CORRECTNESS OF PROGRAMS

I Test, in un programma concorrente, non bastano: perchè ad ogni esecuzione di un test viene scelto uno dei scenari possibili di esecuzione che dipendono dall'architettura del calcolatore, situazione in cui si trova il sistema operativo etc...

- Checking correctness for sequential programs
 - unit testing based on specified **input** and expecting some specified **output**
 - diagnose, fix, rerun cycle
 - re-running a program with the same input will always give the same result
- Nella Concurrent programming new (challenging) perspective
 - **the same input can give different outputs**, depending on the scenario...
 - some scenarios may give correct output while others do not
 - > we can't debug a concurrent program in the normal way
 - each time you run the program we will likely get a different scenario

Dipendenza dallo Scenario: Il risultato non dipende solo dai tuoi dati, ma da come il sistema operativo decide di alternare i processi.
Heisenbug: Un errore appare una volta, ma quando provi a riavviare il programma per studiarlo... sparisce! Questo perché al secondo avvio il "timing" è leggermente diverso e lo scenario incriminato non si verifica.

- Needs of different kind of approaches

↓
formal analysis, *model checking*
↓
based on abstract models

Quindi, dobbiamo usare metodi matematici:

- Analisi Formale: Dimostrare logicamente che, indipendentemente dall'ordine delle azioni, le proprietà di correttezza (es. "due persone non prelevano più del saldo") siano sempre rispettate.
- Model Checking: Usare un software che esplora tutto il diagramma degli stati (quello che abbiamo visto prima). Il Model Checker non "esegue" il programma una volta, ma percorre sistematicamente ogni possibile scenario alla ricerca di quello che porta a un errore.

CORRECTNESS OF CONCURRENT PROGRAMS

- The correctness of (possibly non-terminating) concurrent programs is defined in terms of **properties of computations** degli scenari
 - cioè conditions that must be verified in every possible scenarios
- Two type of correctness properties
 - 1 – **safety** property
 - 2 – **liveness** property

1

SAFETY PROPERTIES

- The property must be **always** true Una proprietà P è di Safety se P è vera per ogni possibile stato in ogni scenario
 - i.e. affinché una proprietà di safety P sia valida for a safety property P to hold, it must be true in every scenario state of every computation Un invariante di computazione
 - expressed as ~~invariants of a computations~~
- Typically used to specify that “bad things” should never happen
 - mutual exclusion
 - no more than one process is ever present in a critical region (vado a controllare in ogni stato se in ogni sezione critica ci sono più di uno stato)
 - no deadlock
 - no process is ever delayed awaiting an event that cannot occur (non ci devono essere stati di deadlock: stato da cui non posso più eseguire transizioni perché non ce ne sono)
 - etc etc

2 LIVENESS (OR *PROGRESS*) PROPERTIES

- The property must **eventually become true** → prima o poi
– i.e. affinché una proprietà di liveness P sia valida for a liveness property P to hold, it must be true that scenario in every computation there is some state in which P is true
Prima o poi, in uno stato di ogni scenario quella condizione/proprietà si verifica (es: JUnit Testing). Non basta vedere solo gli stati, ma bisogna verificare tutti i possibili scenari che prima o poi quella proprietà diventi vera.
- Typically used to specify that “good things” eventually happen
 - no starvation
 - a process ottiene finalmente finally gets the resource it needs (CPU time, lock)
 - no dormancy
 - a waiting process is finally awakened
 - reliable communication
 - a message sent by one process to another will be received
 - etc etc

FAIRNESS

- A liveness property ^{che sostiene che} ~~which holds that~~ something good happens infinitely often → Mentre una generica proprietà di liveness dice che un evento positivo accadrà "prima o poi" (eventually), la fairness è una forma di liveness più forte: richiede che l'evento positivo continui a ripetersi all'infinito durante l'esecuzione del programma.
- Main example
 - a process activated infinitely often during an application execution, each process getting a fair turn
 - i.e. an action that can be executed, eventually will be executed
 - requirement on the scheduling → prima o poi
La fairness non è una caratteristica che puoi scrivere direttamente nel codice del tuo programma; è invece un requisito imposto allo scheduler (il componente del sistema operativo o della JVM che decide quale thread mandare in esecuzione).
- So programs can have different liveness behavior depending on precisely how their instructions are interleaved
 - how instructions are interleaved is a result of a **scheduling policy**. → Di conseguenza, per garantire che una proprietà di liveness sia soddisfatta, siamo costretti a ipotizzare e richiedere un livello minimo di fairness da parte della politica di scheduling. Senza queste assunzioni di fairness, è matematicamente impossibile dimostrare che un programma concorrente farà progressi.

FAIRNESS & SCHEDULING POLICIES

Sono tre livelli crescenti di fairness che definiscono quanto uno scheduler sia "giusto" nel concedere la CPU ai vari processi.

- **Unconditional Fairness**

Garanzia che si applica ad azioni senza condizioni (sempre eseguibili).

- a scheduling policy is unconditionally fair if every unconditional atomic action that is eligible is executed eventually

prima o poi

pronta per essere eseguita, viene eseguita

Definizione: Ogni azione atomica pronta per essere eseguita verrà, prima o poi, scelta dallo scheduler.

Concetto chiave: Lo scheduler non può ignorare un processo all'infinito. Se il processo ha qualcosa da fare, riceverà il suo turno.

- **Weak Fairness**

- a scheduling policy is weakly fair if it is unconditionally fair and every eligible conditional atomic action whose condition becomes and remains true, is executed eventually

che è pronta per essere eseguita

la cui

prima o poi

- **Strong Fairness**

- a scheduling policy is strongly fair if it is unconditionally fair and every eligible conditional atomic action whose condition becomes true infinitely often (infinitely many times), is executed eventually

che è pronta per essere eseguita

la cui

prima o poi

Definizione: Se una condizione diventa vera infinitamente spesso (anche se per pochissimo tempo ogni volta), lo scheduler garantisce che l'azione verrà eseguita.

Il punto di forza: Lo scheduler è "attento". Anche se l'opportunità è intermittente (va e viene), prima o poi "beccherà" il momento giusto per far avanzare il processo.

In breve: Previene la starvation anche in sistemi molto caotici dove le risorse vengono prese e rilasciate continuamente.

Definizione: Se una condizione diventa vera e rimane vera fino all'esecuzione, lo scheduler garantisce che l'azione verrà eseguita.

Il limite: Se la condizione diventa vera solo per brevi intervalli e poi torna falsa (lampeggia), lo scheduler "pigro" potrebbe non accorgersene mai e il processo resta bloccato.

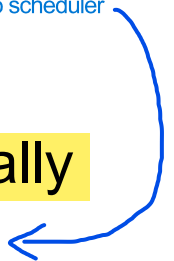
UNCONDITIONALLY FAIR SCENARIO

- def. **unconditionally fair scenario**

Un scenario è considerato "fair" se un processo ha un'istruzione pronta per essere eseguita (continually enabled), lo scheduler deve prima o poi farla apparire nella sequenza.

- a scenario is (unconditionally) fair if at any state in the scenario a statement that is continually enabled eventually appears in the scenario

Significa che non accettiamo come "validi" quegli scenari in cui un processo viene ignorato per sempre mentre gli altri continuano ad essere eseguiti. Se un'istruzione può essere eseguita, in uno scenario fair verrà eseguita.



```
integer n := 0
boolean flag := false
```

p	q
p1: while flag = false p2: n := 1 - n	q1: flag := true

- Does this algorithm necessarily halt?

L'algoritmo si ferma?

- yes if we assume only fair scenarios
 - if we allow only fair scenario, then eventually an execution of q1 must be included in every scenario

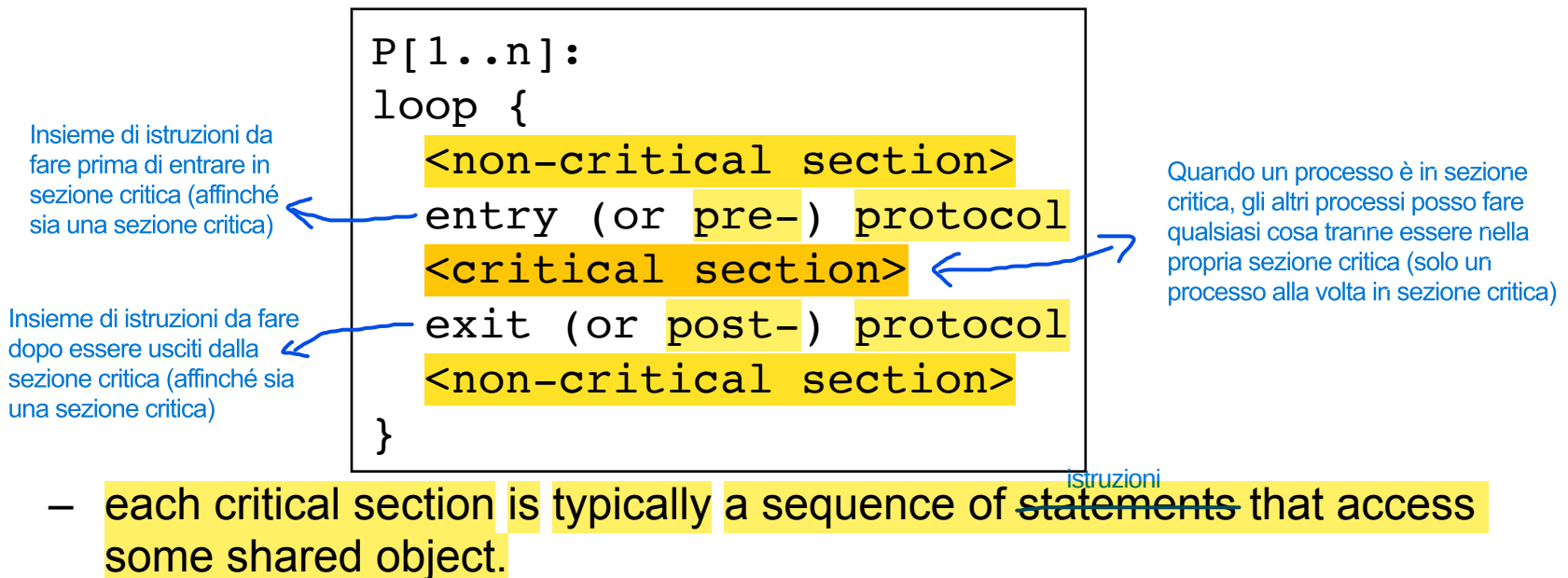
- **the non-terminating scenario is not fair**

Esempio: in un sistema distribuito non viene mai eseguito q perché è down

CRITICAL SECTION PROBLEM

Una sezione critica è una PARTE DI CODICE nei programmi concorrenti che accede a risorse condivise e deve essere eseguita da un solo processo o thread alla volta per evitare data corruption o contraddizioni. Questo requisito è noto come mutual exclusion

- Introduced by Dijkstra in 1965, the critical section problem can be considered one of the most important and well-studied problem in concurrent programming, in the context of process competition
- Problem definition**
 - ciascuno dei quali esegue
N processes, each executing in an infinite loop, a sequence of Istruzioni statements that can be divided in 2 subsequences: the *critical section* (CS) and the *non-critical section* (NCS)



CRITICAL SECTION PROPERTIES

- Our task is to design *entry and exit protocols* that satisfy the following properties:
 - **mutual exclusion** (un processo alla volta nella propria sezione critica)
 - statements from the critical sections of two or more processes must not be interleaved
 - **freedom from deadlock**
 - if some processes are trying to enter their critical sections, then one of them must eventually succeed (almeno uno dei processi che ha fatto richiesta prima o poi entrerà nella propria sezione critica)
 - **freedom from individual starvation**
 - if any process tries to enter its critical section, then that process must eventually succeed (prima o poi quel processo entrerà nella propria sezione critica)
 - *bounded waiting* property
- Any proposed solution must satisfy also the progress property for the CS
 - once a process starts to execute the statements of its critical section, it must eventually finish.
- The NCS need not to progress (posso rimanere all'infinito in NCS: quindi non vale la progress property per NCS)
 - we can have infinite loops or termination outside the CS

Critical Section

CS

prima o poi, si esce dalla sezione critica

prima o poi

eventually finish.

CONCRETE PROBLEMS BASED ON CRITICAL SECTIONS

- The critical section problem is intended to model a system that performs complex computation, but occasionally need to access data or resources (e.g. hardware) that are shared by several process.
- Example
 - a check-in-kiosk at an airport, accessing to a central database of passengers

CRITICAL SECTION SOLUTION: ALGORITHMS VS. MECHANISMS

- The CS problem can be easily solved by using basic mechanisms such as *semaphores* or *locks*, that must be provided by the concurrent machine
– next modules


Meccanismi di alto livello che rendono, insieme di istruzioni non-atomiche, atomiche (che semplificano la soluzione rispetto a quella puramente algoritmica)
- In this module we focus instead on a purely algorithmic solution (Soluzione articolata)
– without using high-level mechanism
– using solely basic atomic statements (e.g. load and store)


Costrutti sequenziali di base (while, if, ..) ed Assegnamento Atomico di variabili

CS FOR TWO PROCESSES

Ricorda assunzione: istruzioni etichettate con etichetta vengono eseguite atomicamente

- First we will analyse the problem considering 2 processes P and Q sharing some global variables

Critical section problem	
Global variables	
p	q
Local variables	Local variables
loop forever	loop forever
non-critical section	non-critical section
pre-protocol	pre-protocol
critical section	critical section
post-protocol	post-protocol

- The protocols may require local or global variables
 - we assume that no variables used in the CS and NCS are used in the protocols

1 FIRST ATTEMPT

- Let's consider a first possible solution

First attempt	
Integer turn $\leftarrow$ 1	
p	q
<pre>loop forever p1: non-critical section p2: await turn = 1 Pre-Protocol di p p3: critical section p4: turn $\leftarrow$ 2 Post-Protocol di p</pre>	<pre>loop forever q1: non-critical section q2: await turn = 2 Pre-Protocol di q q3: critical section q4: turn $\leftarrow$ 1 Post-Protocol di q</pre>

await sarebbe: while (turn != 1)

- The statement **await turn = 1** is an implementation independent notation for a statements that **waits until the condition turn = 1 becomes true.**
 - this can be implemented (albeit inefficiently) by a busy-wait loop that does nothing until the condition is true.

1 CHECKING CORRECTNESS

- Per dimostrare In order to prove the correctness of the solution we construct the state diagram of the concurrent program
 - the algorithm must satisfy the three properties (Mutual Exclusion, Freedom from Deadlock and Freedom from Individual Starvation)
- To check correctness properties is only necessary to examine the set of reachable states and the transitions among them
 - states are triple $\langle p_i, q_j, \text{turn} \rangle$ where:
 - p_i = current control pointer for process P (ovvero quale istruzione P deve eseguire adesso o in quale blocco si trova)
 - q_j = current control pointer for process Q (ovvero quale istruzione Q deve eseguire adesso o in quale blocco si trova)
 - turn = content of turn variable
 - the diagram is constructed incrementally starting from the initial state and considering what the potential states are
 - 16 states
- Describing the mutual exclusion property la proprietà di correttezza della mutua esclusione è soddisfatta
 - (non deve esserci uno o più stati in cui ho sia p3 sia q3 contemporaneamente, a prescindere dal valore di turn) the mutual exclusion correctness property holds if the set of all accessible states does not contain a state of the form $\langle p_3, q_3, _ \rangle$.

1

REDUCING THE NUMBER OF STATES

- For studying correctness, we can reduce the number of states without altering the semantics of the program
 - by removing unessential statements
- Whatever statements are executed by the CS and in the NCS are totally irrelevant to the correctness of the synchronisation algorithms, so they can be removed (vengono lasciati i Protocolli Pre e Post CS)

Semplifico il modello rimuovendo le istruzioni non essenziali

First attempt (abbreviated)	
Integer turn $\leftarrow$ 1	
p	q
loop forever p1: await turn = 1 p2: turn $\leftarrow$ 2	loop forever q1: await turn = 2 q2: turn $\leftarrow$ 1

Anche qui, ho starvation (es: scelgo solo q1 e così p non entra mai in CS)

- The mutual exclusion properties holds if there are not states of of the kind $\langle p_2, q_2, _ \rangle$ (non ci devono essere stati dove entrambi i processi vogliono assegnare il turn a 1 o 2)
 - building the new state diagram => only 4 states

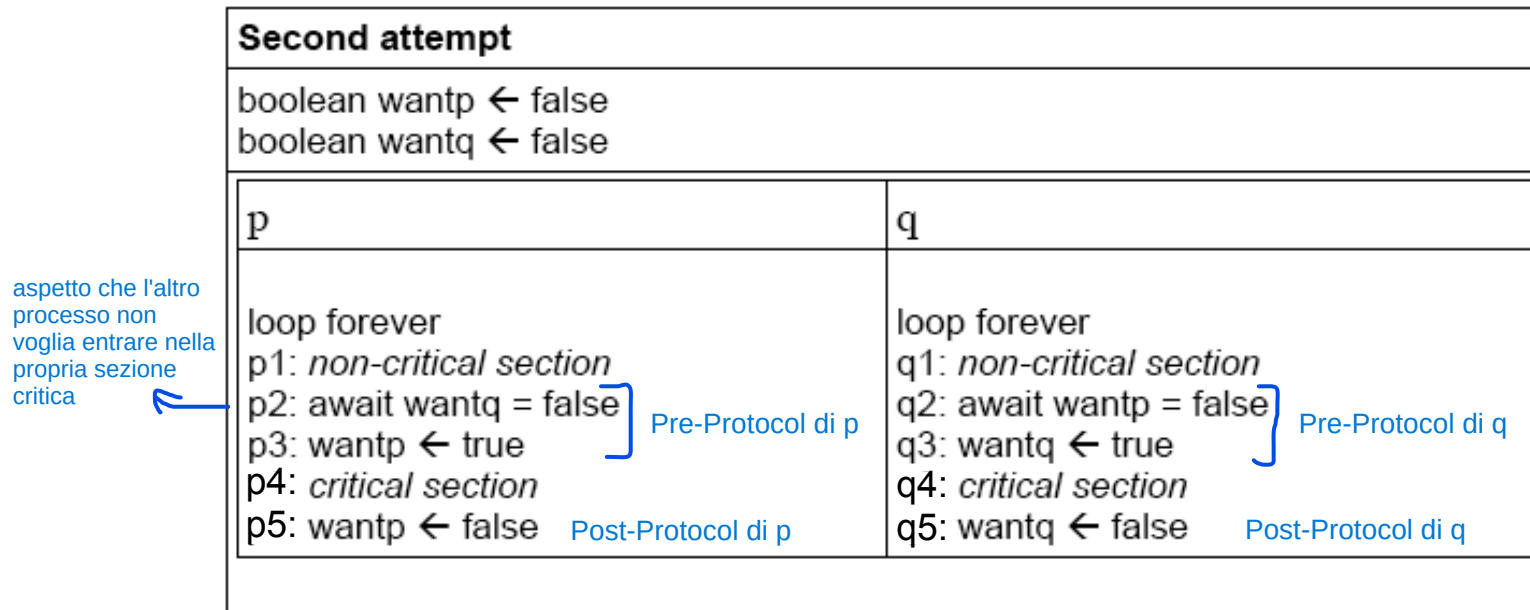
1

CHECKING CORRECTNESS (2)

- To check the correctness we have to check that the three properties ^{sono valide} hold
 - mutual exclusion is satisfied
 - there are not scenarios including states of the kind $\langle p_2, q_2, _ \rangle$
 - freedom of deadlock is satisfied
 - if some processes are trying to enter the CS (executing the await), then one eventually succeeds
 - freedom of starvation is **not satisfied**
 - a process that wants to enter in CS (pre-protocol) can starve while waiting a process which engaged an infinite loop inside its NCS (where there is not the progress property)
- ^{La variabile} Turn function as permission resource to enter the CS
 - if the process holding the permission remains indefinitely in its non-critical section, the other process will never receive the resource and will never enter its critical section
 - both processes sets and tests a single global variable: if one process dies, the other is blocked.
- Therefore the solution is *not correct*

2 SECOND ATTEMPT

- We introduce separate want variables intended to indicate when a process is in its CS (intenzione di p e q di entrare nella propria sezione critica)



Se sono in p2 e q2 contemporaneamente, significa che entrambi non vogliono entrare, ma in realtà non hanno ancora impostato a true la loro voglia di entrare (quindi alla fine i want diventano entrambi contemporaneamente a true e quindi non viene soddisfatta la mutua esclusione)

- Mutual exclusion: not having scenarios with p_4 and q_4

2 SECOND ATTEMPT ABBREVIATED

- Removing irrelevant statements..

Second attempt: Abbreviated	
boolean wantp $\leftarrow$ false boolean wantq $\leftarrow$ false	
p	q
loop forever p1: await wantq = false p2: wantp $\leftarrow$ true p3: wantp $\leftarrow$ false	loop forever q1: await wantp = false q2: wantq $\leftarrow$ true q3: wantq $\leftarrow$ false

- Mutual exclusion: not having scenarios with p_3 and q_3

CORRECTNESS OF THE 2nd ATTEMPT

- By constructing the state diagrams it is simple to see that the mutual exclusion property is not satisfied
 - the state $\langle p_3, p_3, \text{true}, \text{true} \rangle$ is reachable.
- The problem is due to the non-atomicity of the pre-protocol and post-protocol

3 THIRD ATTEMPT...

- Previous problem can be solved by considering the await statement part of the CS and moving the assignment before the await → (ora viene swappato rispetto a prima: ora si mette a true la propria intenzione e poi si controlla che quello contro non voglia entrare: però ora ho deadlock)

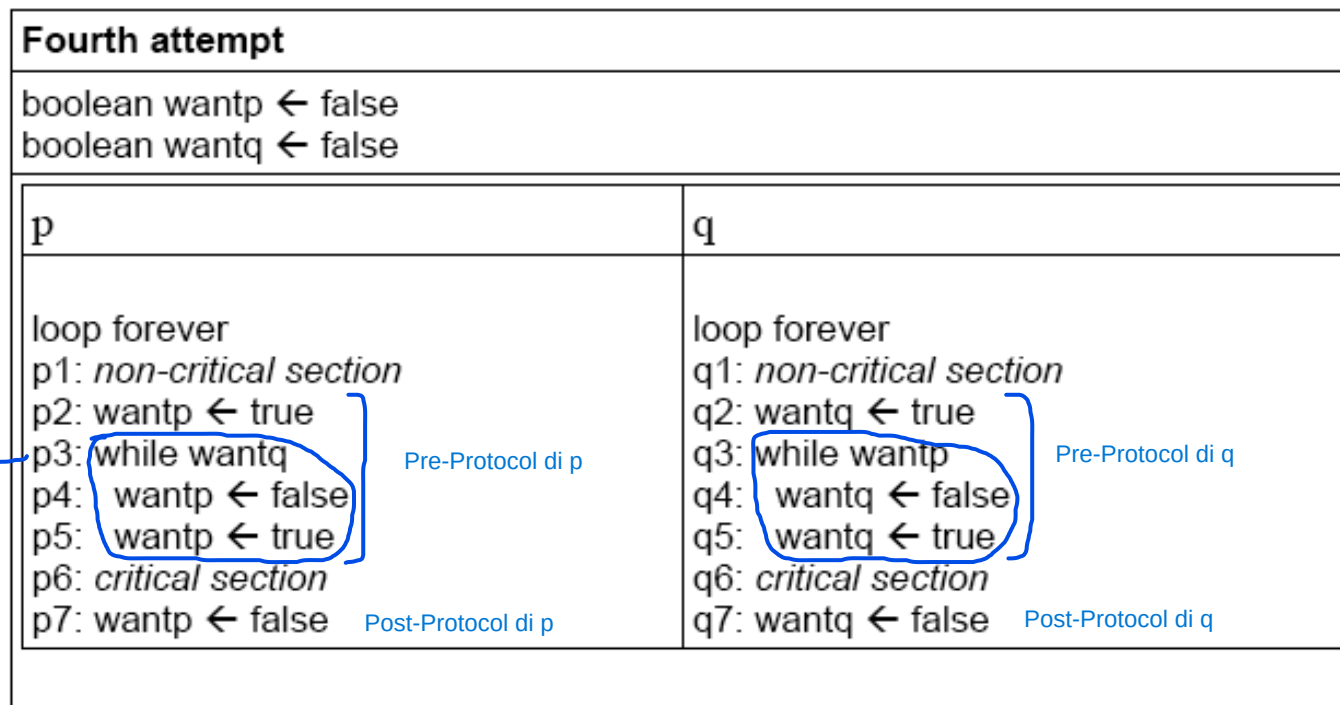
Third attempt	
boolean wantp ← false boolean wantq ← false	
p	q
loop forever p1: <i>non-critical section</i> p2: wantp ← true p3: await !wantq p4: <i>critical section</i> p5: wantp ← false	loop forever q1: <i>non-critical section</i> q2: wantq ← true q3: await !wantp q4: <i>critical section</i> q5: wantq ← false

↓
i due processi si bloccano: i due want non diventeranno mai false perchè entrambi si bloccano su await

- Correctness of the 3rd attempt
 - the mutual exclusion problem is solved, however the solution suffers of possible deadlocks (se faccio prima p2 e poi q2, si va in deadlock)
 - both processes are trying to enter in their CS but neither will ever do
 - more precisely, this is a *livelock* (busy waiting)

4 FOURTH ATTEMPT...

- **Deadlock occurs** when both processes simultaneously insist on entering their CS
 - we can solve the problem by requiring a process to give up its intention to enter its CS if it discovers that it is contending with the other process



- **Correctness of the 4th attempt**
 - the livelock is solved, but there is still the possibility of starvation, in the case of perfect interleaving in executing while...
(entrambi vedono sempre che l'altro vuole entrare in CS e quindi nessuno dei due mai va in CS perchè avviene in perfetto interleaving l'esecuzione del while)

DEKKER'S ALGORITHM

- The problem can be solved with a simple variation of the 4th algorithm, ~~requiring that *the right to insist on entering*~~ ^{che prevede che il diritto di insistere per entrare} - instead of the right to enter - is explicitly passed between the processes by means of the variable turn
 - this solution was found for the first time by the dutch mathematician T.J. Dekker, in 1965
 - Dekker's algorithm for solving the CS problem can be then considered a combination of the first and fourth attempt

5 A SOLUTION: DEKKER'S ALGORITHM

Dekker's algorithm

Algo di Dekker funziona solo per due processi
(bisogna estendere il ragionamento a 2+ processi)

```
boolean wantp ← false
boolean wantq ← false
integer turn ← 1
```

p

q

```

loop forever
p1: non-critical section
p2: wantp ← true
p3: while wantq
p4:   if turn = 2
p5:     wantp ← false
p6:     await turn = 1
p7:     wantp ← true
p8: critical section
p9: turn ← 2
p10: wantp ← false
    
```

Se è il turno di q, lascio entrare q in CS (questo risolve il problema di Starvation della prima soluzione)

Pre-Protocol di p

Post-Protocol di p

```

loop forever
q1: non-critical section
q2: wantq ← true
q3: while wantp
q4:   if turn = 1
q5:     wantq ← false
q6:     await turn = 2
q7:     wantq ← true
q8: critical section
q9: turn ← 1
q10: wantq ← false
    
```

Pre-Protocol di q

Post-Protocol di q

p vuole entrare in CS

se anche q vuole entrare (conflitto), se no salto il while ed entro direttamente

Dopo che è il turno di p, rimetto la voglia di entrare in CS

- Dekker's algorithm is correct
 - it satisfies mutual exclusion, and it is free from deadlock and starvation.

6

PETERSON'S ALGORITHM

- Other solutions have been proposed through time, improving Dekker's one:
 - Peterson (1981), Manna-Pnueli, Doran-Thomas
- Peterson's solution is more concise, by collapsing the two await statements into one with a compound condition

```
boolean wantp := false;
boolean wantq := false;
integer turn := 1;
```

p

```
loop forever
p1: NCS
p2: wantp := true
p3: turn := 2
p4: await !wantq || turn = 1
p5: CS
p6: wantp := false
```

Pre-Protocol di p

Post-Protocol di p

q

```
loop forever
q1: NCS
q2: wantq := true
q3: turn := 1
q4: await !wantp || turn = 2
q5: CS
q6: wantq := false
```

Pre-Protocol di q

Post-Protocol di q

Aspetto che l'altro processo non voglia
entrare in CS oppure che sia il mio turno

INTRODUCING COMPOUND ATOMIC STATEMENTS

- Dekker's algorithm works on any architecture, providing just load and store as atomic statements.
- Actually, the algorithm and – more generally – the CS problem and mutual exclusion problems can be greatly simplified if we can exploit more complex atomic statements
 - directly provided by the concurrent machine
- Main examples are the *test-and-set*, *exchange*, *fetch-and-add*, *compare-and-swap*
 - test-and-set is an atomic statement defined as the execution of the two following statements with no possibility of interleaving

Per chiarire la dinamica in breve:

- Legge il vecchio valore della variabile condivisa x.
- Scrive il nuovo valore in x (tipicamente true o 1 per acquisire un lock).
- Restituisce il valore letto precedentemente, che ora si trova in r.



```
test-and-set (x, r):  
< r := x, x := 1 >
```

Test-and-set viene usata per scrivere in una locazione di memoria e restituire il suo vecchio valore come una singola operazione atomica (non interrompibile). Se diversi processi possono accedere alla stessa area di memoria, e se un processo sta eseguendo una test-and-set, nessun altro processo può iniziare un'altra test-and-set finché il primo processo non ha terminato la propria.

- a commonly used notation for specifying atomicity is to put angled bracket around the group of statements

CRITICAL SECTION WITH TEST-AND-SET

test-and-set (x,r):
< r := x, x := 1 >

Questa soluzione funziona con N processi

integer lock := 0; Variabile Condivisa	
p	q
integer was_locked loop forever p1: NCS repeat p2: test_and_set(lock, was_locked) p3: until was_locked = 0 p4: CS p5: lock := 0 Ripeto fino a che il valore precedente di Lock è 0: prendo il valore di lock, lo metto in was_locked e poi setto lock ad 1; se was_locked è ad 1 significa che un altro processo è in CS, se a 0, allora nessuno è in CS e posso entrare in CS Pre-Protocol di p	integer was_locked loop forever q1: NCS repeat q2: test_and_set(lock, was_locked) q3: until was_locked = 0 q4: CS q5: lock := 0 Pre-Protocol di q

Quando viene eseguita transizione p2/q2, nel LTS mantiene l'esecuzione di test_and_set in maniera atomica

LOCKS

Un lock è un meccanismo di sincronizzazione fondamentale nella programmazione concorrente, utilizzato per regolare l'accesso esclusivo a una risorsa condivisa da parte di più processi. L'obiettivo principale di un lock è garantire la Mutua Esclusione: assicurare che un solo processo alla volta possa eseguire la propria Sezione Critica (CS) o accedere a un determinato blocco di memoria.

Sfruttando l'istruzione atomica composta

- ~~By exploiting compound atomic statement~~, we can easily realize a basic *lock* mechanism
 - simple solution for the CS problem
 - protecting critical section of code
- Two stages / atomic operations
 - *acquiring* the lock
 - *releasing* the lock

Sotto certe assunzioni, possiamo considerare 'acquire' un'istruzione atomica (anche se contiene due istruzioni: p2&p3 o q2&q3)

lock sharedlock;		Utilizzabile per N processi
p	q	
loop forever p1: NCS p2: acquire(sharedlock) p3: CS p4: release(sharedlock)	loop forever q1: NCS q2: acquire(sharedlock) q3: CS q4: release(sharedlock)	

ATOMIC STATEMENT IN JAVA:

synchronized

Acquire di un Lock (equivalente di un test_and_set)

- Atomic statements in Java can be implemented by means synchronized blocks using the same shared lock object

```
// process (thread) A
...
synchronized (lock) {
    <statement a>
    <statement b>
    ..
}
...
```

```
// process (thread) B
...
synchronized (lock) {
    <statement c>
    <statement d>
    ..
}
...
```

- in this way the sequence of actions a.b and c.d are executed without interleaving the individual actions
 - there cannot exist scenarios with sequence like a.c.b.d or c.a.d.b

CS SOLUTION WITH TICKETS: BAKERY ALGORITHM (TIE-BREAKER)

- An alternative solution to CS problem for N-process ~~accounts for~~ ^{consiste nel} ~~introducing~~ ^{introdurre} a *ticket* to establish the turn of a process

N-process solution for CS with tickets

```
int num ← 1  numero di ticket prossimo da staccare
int next ← 1  prossimo numero da servire
turn[1:n] ← [0,0,0,...]  rotolo dei ticket (ad ogni indice, corrisponde un processo con il suo ticket)
```

p[i] Processo i-esimo

```
loop forever
p1: NCS
p2: < turn[i] ← num; num ← num + 1 >  Operazione atomica: prendo il ticket e viene incrementato il numero del prossimo ticket
p3: await turn[i] = next  Aspetta il suo turno
p4: CS
p5: next ← next + 1  Finita la sezione critica, avanza next (prossimo numero da servire)
```

- ^{Possibili criticità} ~~Potential shortcomings~~: arithmetic overflow for turn and num
 - monotonically growing

ABOUT CORRECTNESS: VERIFICATION VS. TESTING

Un Fault è ciò che il programmatore cerca e corregge nel codice (il bug statico); un Failure è ciò che l'utente finale sperimenta e subisce (il crash o malfunzionamento dinamico).

• Cercare la presenza di bug in uno scenario, che è quello di test

Testing

- attività volta all'individuazione dei Fallimenti
- **activity looking for failures**
 - revealing faults

Verification

Verificare se una proprietà di correttezza è vera in tutti i possibili scenari di esecuzione

- checking if the system satisfies its specification

- “**have we built the system right?**”

Validation

Interessa sapere se il sistema è stato costruito come si aspettava lo stakeholder (rispettare requisiti)

- checking if the system satisfy customer's expectation

- “**have we built the right system?**”

Bug

Fault

La presenza dell'anomalia nel codice.

- a **manifestation** of an error
 - error = (human) action **that** produces an incorrect result

Fallimento

Failure

L'evento visibile in cui il sistema smette di funzionare correttamente.

- when a **fault** is encountered, it may result in a failure
 - a failure may be caused by many **faults**
 - a **fault** can cause many failures

Error (l'azione umana errata) -> Fault (il bug nel codice) -> Failure (il crash visibile all'utente).

LIVNESS AND SAFETY TESTS

Si intende anche la
Verifica, non solo tests

- Two kinds of tests
 - **Tests of safety** (Qualcosa che deve valere sempre)
 - verify that a class or system behavior conforms to its specification
 - usually taking the form of testing **invariants**
 - typically using *assertions*
 - **Tests of liveness** (Qualcosa che prima o poi diventa qualcos'altro)
 - progress / non-progress tests
 - harder to set up
- **Problems & limits of testing**
 - typically checking only a subset of the scenarios
 - “*heisenbugs*” phenomenon
 - test code can introduce timing or synchronisation artefacts that can mask bugs

Un heisenbug è un difetto software che scompare o cambia comportamento quando si tenta di studiarlo, osservarlo o correggerlo (debugging). Il nome deriva dal principio di indeterminazione di Heisenberg, poiché l'atto di monitorare il programma (es. inserendo log o usando un debugger) altera l'ambiente di esecuzione, facendo sparire il bug.

TESTING VS. VERIFICATION

- Testing is aimed at verifying that a (correctness) property holds for some selected scenario
 - *testing reveals the presence of errors, not their absence*
- Verification is aimed at verifying that a property holds **for all possible scenarios (traces)**
 - need of *formal* techniques

FORMAL METHODS

VERIFICATION

- Two principal (classes of) formal techniques:
 - **model checking**
 - where verification is done by generating one by one all the states of the systems and by checking the properties state by state
 - can be automated by model checkers tools
 - **inductive proofs of invariants**
 - Dimostrazioni per induzione degli invarianti ~~invariant properties~~ le proprietà degli invarianti vengono dimostrate are proved by *induction* rispetto agli ~~over the~~ states of the system
 - can be automated by tools called deductive systems
- Both techniques rely on some kind of *formal language / calculus to specify correctness properties*

CORRECTNESS PROPERTIES IN PROPOSITION CALCULUS

- ^{Con la Logica proposizionale} ~~With propositional calculus~~, correctness properties are expressed as *logic formulae* that must be true ^{affinché la proprietà sia verificata} ~~in order to verify the property~~ in some state of the system
 - formulae are assertions obtained by composing propositions through logic connectors
 - and, or, not, implications, equivalence
- In our case propositions are about *the values of the variables* and *of the control pointers* during an execution of a concurrent programming
 - e.g. given the boolean variable `wantp`, an atomic proposition (assertion) `wantp` is true in a certain state if and only if the value of the variable `wantp` is true in that state
- ^{etichetta di un'istruzione} ~~Each label of a statement~~ of a process will be used as an atomic proposition whose interpretation is "the control pointer of that process is currently at that label"
 - e.g. `p1` proposition asserts that the control pointer of the process `p` is at the label `p1`

Problema: la logica proposizionale non è adeguata per i nostri scopi perchè non ci permettono di esprimere, dato il programma, proprietà che si esprimono nel tempo (riguardante la dinamica del tempo)

Modelling Concurrent Program Execution 60

AN EXAMPLE: MUTUAL EXCLUSION

Third attempt	
boolean wantp $\leftarrow$ false boolean wantq $\leftarrow$ false	
p	q
loop forever p1: <i>non-critical section</i> p2: wantp $\leftarrow$ true p3: await !wantq p4: <i>critical section</i> p5: wantp $\leftarrow$ false	loop forever q1: <i>non-critical section</i> q2: wantq $\leftarrow$ true q3: await !wantp q4: <i>critical section</i> q5: wantq $\leftarrow$ false

Con la formula logica proposizionale, non riesco a catturare il SEMPRE

- Formula $p_4 \wedge q_4$
 - is true if both control pointers of the processes are in the critical section
- if it exists some state in which this formula is true, then it means that the mutual exclusion property is **not** satisfied
- > dually, a program *satisfies the mutual exclusion property* if the formula $\neg(p_4 \wedge q_4)$ is true for every possible state of every scenario

TEMPORAL LOGIC

È un sistema logico che permette di esprimere come il valore delle verità cambia nel tempo. Invece di dire solo "P è vero", possiamo dire "P sarà vero in futuro" o "P è sempre stato vero". Nel contesto della programmazione concorrente, è fondamentale perché ci permette di descrivere il comportamento del programma lungo gli scenari di esecuzione.

- Processes and systems change their state over the time, and then also the interpretation of formulae about their state can change over the time
 - > we need a formal language/~~calculus~~^{logic} that would take this aspect into the account (cosa che la logica proposizionale non fa: non considera l'evoluzione del sistema nel tempo)
 - > *temporal logic* is one of the most basic and popular one
 - Amir Pnueli, "The Temporal Logic of Programs", 18th Annual Symposium on Foundations of Computer Science, 1977
- The **temporal logic** is a formal logic obtained by adding temporal operators to propositional or predicate logic → Introducono degli operatori che permettono di esprimere proprietà di sistemi che evolvono nel tempo
 - **Linear Temporal Logic (LTL)**
 - to ~~express~~^{per esprimere} properties that must be true (~~at a state~~^{in uno stato}) for every possible scenario
 - linear / discrete model of time
 - **Branching Temporal Logics**
 - to express properties that must be true in some or all scenarios starting from a state
 - an example: **CTL (computational tree logic)**

LTL: TEMPORAL OPERATORS

Vedremo LTL solo con logica proposizionale

- LTL is based on two basic temporal operators: *always* and *eventually*

Proprietà
sempre vera

– **box** or **always** temporal operator: $\Box p$

- the formula $\Box p$ is true in a state s_i of a computation ^{scenario} if and only if the formula p is true in *all* state s_j with $j \geq i$

Proprietà p deve essere vera in tutti i possibili stati dallo stato s_i in poi, per ogni scenario

– synonym: $\Box p = G p$ (Globally p)

- the always operator can be used then to specify *safety properties*, because it specifies what must be always be true

Proprietà
vera almeno
una volta

– **diamond** or **eventually** temporal operator: $\Diamond p$

- the formula $\Diamond p$ is true in a state s_i of a computation ^{scenario} if and only if the formula p is true in *some* state s_j with $j \geq i$

Per ogni scenario, deve esistere almeno uno stato in cui p sia vera

– synonym: $\Diamond p = F p$ (Finally p)

- the eventually operator is used to specify *liveness properties*, because it specifies something that eventually be true

BASIC PROPERTIES

- Reflexivity Riflessività
 - $\Box p \rightarrow p$
 - $p \rightarrow \Diamond p$
- Duality Dualità
 - $\neg \Box p \rightarrow \Diamond \neg p$ Se non è sempre vero p, allora prima o poi p diventerà falso
 - $\neg \Diamond p \rightarrow \Box \neg p$ Se non è vero che prima o poi succederà p, allora è sempre vero che p è falsa
- Sequences of operators
 - $\Diamond \Box p$ Prima o poi esiste uno stato da cui p è sempre vera
 - $\Box \Diamond p$ Sempre vera che prima o poi si verifichi p (si vuole che p diventi vera più di una volta, non solo una volta: caso di loop)

DEDUCTION WITH TEMPORAL LOGICS

Partendo dal codice, usi la logica temporale per "dimostrare" matematicamente che il programma rispetterà la sua specifica, senza doverlo mai far girare su un computer reale.

è un sistema formale di logica deduttiva

- Temporal logic ~~is a formal system of deductive logic~~ with its own axioms and rules of inference
 - tradurre matematicamente il comportamento di un programma (cosa fa, come si evolve e come reagisce) per eliminare ogni possibile ambiguità o interpretazione soggettiva.
 - it can be used to formalise the semantics of concurrent programs and used to rigorously ~~prove~~ ^{dimostrare} correctness properties of programs
- Example of theorems in TL
 - $(\Diamond \Box p \wedge \Diamond \Box q) \rightarrow \Diamond \Box (p \wedge q)$ is true
 - $(\Box \Diamond p \wedge \Box \Diamond q) \rightarrow \Box \Diamond (p \wedge q)$ is false

Perché ci può essere un caso in cui p è vera e q falsa o viceversa

Perché è meglio del Model Checking (in certi casi)?

Mentre il Model Checking esplora il grafo degli stati (ed è limitato dalla memoria del PC), la deduzione con logica temporale è puramente simbolica: Puoi dimostrare che un algoritmo di ordinamento funziona per una lista di N elementi, dove N può essere anche un miliardo o infinito. Il Model Checking dovrebbe testare il caso con 1 elemento, poi con 2, poi con 3... e si fermerebbe molto presto.

SPECIFYING SAFETY PROPERTIES

- Box operator can be used to specify safety properties
 - as properties that must be always true
- $\Box p$, where $p = \neg q$ and q is the description of a *bad* state
 - an example: mutual exclusion in CS problem

First attempt	
Integer turn $\leftarrow 1$	
p	q
loop forever p1: <i>non-critical section</i> p2: await turn = 1 p3: <i>critical section</i> p4: turn $\leftarrow 2$	loop forever q1: <i>non-critical section</i> q2: await turn = 2 q3: <i>critical section</i> q4: turn $\leftarrow 1$

- mutual exclusion property: $\Box \neg (p_3 \wedge q_3)$ Per ogni scenario, in tutti gli stati deve valere la formula

SPECIFYING LIVENESS PROPERTIES

- Diamond operator can be used to specify liveness properties
 - as conditions that eventually will be true
- $\diamond p$, where p is the description of a *good case*
 - an example: progress property (no starvation) in CS problem

First attempt	
Integer turn $\leftarrow 1$	
p	q
loop forever p1: <i>non-critical section</i> p2: await turn = 1 p3: <i>critical section</i> p4: turn $\leftarrow 2$	loop forever q1: <i>non-critical section</i> q2: await turn = 2 q3: <i>critical section</i> q4: turn $\leftarrow 1$

se la proprietà è vera, in questo caso, significa che deve rispettare la proprietà solo nello stato corrente (cioè rispetta solo una volta)

se p2 è vera allora prima o poi p3 vera

– progress property for *current state*: $p_2 \rightarrow \diamond p_3$

In attesa di entrare in sezione critica, prima o poi ci entra

– progress property for *all states*: $\square (p_2 \rightarrow \diamond p_3)$

con box, significa che in qualsiasi stato se capita che vale p2 allora prima o poi p3

NEXT OPERATOR

- Besides always and eventually, another unary operator is defined in LTL, called next: $\circ\varphi$
Vera, in quello stato, se φ è vera nello stato successivo, in ogni scenario possibile
 - $\circ\varphi$ is true if property φ holds in next state, in every possible scenario
 - also represented by X, i.e. $X\varphi$ (from neXt)

BINARY OPERATORS

- *Always* and *Eventually* are unary operators. An example of useful and frequently used binary operator is *Until*
 - **Until operator: $p \text{ U } q$** Prima o poi deve verificarsi q, ma prima che si verifichi q, p deve essere vera (sarebbe p eventually q: cioè q prima o poi diventa vera)
 - $p \text{ U } q$ is true in a state s_i if and only if q is true in some state $s_j, j \geq i$ and p is true in all state $s_k, i \leq k < j$.
 - That is: eventually q becomes true and that p is true until that happens
 - **Weak-Until operator: $p \text{ W } q$** (se q non si verifica mai, p è sempre vera)
 - like Until operator, but formula q is not required to become true eventually. If it does not, p must remain true indefinitely
 - $p \text{ W } q$ = as long as q is false, p must be true

REMARKS

- Always and Eventually operators can be defined in term of until: Con until, posso generare always ed eventually

– $\Diamond p = \text{true} \cup p$ (eventually A becomes true)

– $\Box p = \neg \Diamond (\neg p)$ (A always remains true) p sempre vera è equivalente a dire che non è vero che esiste in futuro uno stato dove p è falsa

p deve diventare vera prima che lo diventi q: 'not q until p' (quando 'p' diventa vera, allora 'q' può diventare vera (diventa vera prima 'p' e poi 'q'): non può farlo prima perchè 'p' non è vera e 'q' deve rimanere falsa per definizione di Until)

OVERTAKING

L'overtaking indica lo scenario in cui un processo viene ripetutamente "superato" da un altro processo nell'accesso alla Sezione Critica

- Consider the following scenario in the CS problem

$$\text{try}_p, \text{try}_q, \text{CS}_q, \text{try}_q, \text{CS}_q, \dots, \text{CS}_q, \text{CS}_p$$

q entra in CS molte
più volte di p

1000 times

- It's not an example of starvation...

- it is true that $\diamond \text{CS}_p$ p entra in sezione critica solo una volta su 1000

- > but it's evident too that freedom from starvation can be a very weak property!

- in some cases we want to ensure that a process would enter its critical section within a reasonable amount of time

K-BOUNDED OVERTAKING PROPERTY

- k -bounded-overtaking property**

Se un processo 'p' fa richiesta di entrare in CS, un altro processo può entrarci al massimo k volte prima di 'p'

- from the time a process p attempts to enter its critical section, another process can enter at most k times before p does

- Example: 3-overtaking

- $\text{try}_p, \text{try}_q, \underline{\text{CS}_q}, \text{try}_q, \underline{\text{CS}_q}, \text{try}_q, \underline{\text{CS}_q}, \underline{\text{CS}_p}$

k riguarda il numero totale di sorpassi che tutti gli altri processi, complessivamente, possono fare su quello specifico processo rimasto in attesa.

- The property can be expressed by the weak until operator W

- example: 1-bounded overtaking

$$\text{try}_p \rightarrow (\neg \text{CS}_q) W (\text{CS}_q W ((\neg \text{CS}_q) W \text{CS}_p))$$

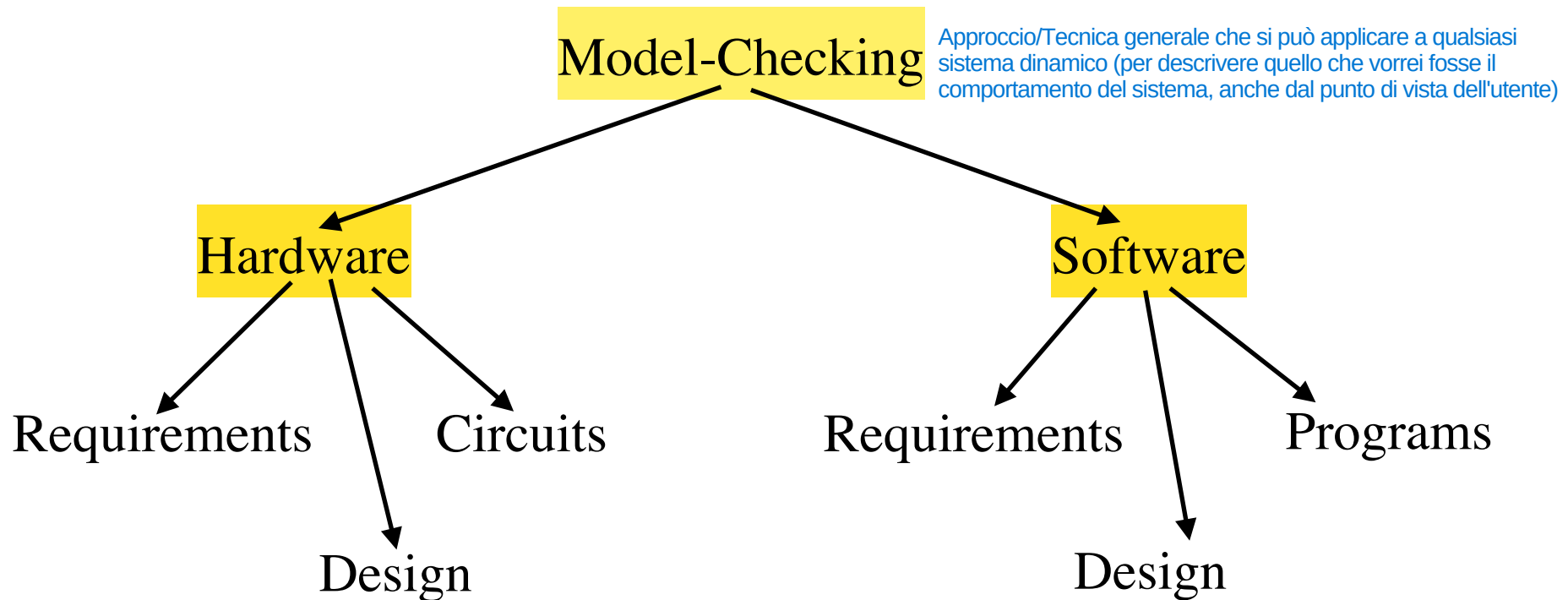
any position satisfying try_p (that is P is in try_p) is followed by an interval in which Q is not in cs, followed by an interval in which Q is in cs, followed by an interval in which Q is again not in cs, which can be terminated only by a state in which P is in cs. It therefore implies that from the time P starts waiting at try_p , there can be at most one continuous interval in which Q is in cs, before P is admitted to cs.

VERIFICATION TECHNIQUES (1/2): MODEL-CHECKING

generazione di tutti i possibili percorsi (nel nostro caso)

- **Model checking** is the most important and used technique for automatically checking correctness properties of concurrent systems
 - invaluable conceptual and practical tool for software engineers
- Strategy based on ~~exhaustively searching the entire state space~~ ^{sulla ricerca esaustiva dell'intero spazio degli stati} of a system and verify if certain properties are satisfied
 - properties as predicates on a system state or states, expressed as a logical specification such as propositional temporal logic formula
 - if the system satisfies the property, the model checker generates a *confirmation* response
 - otherwise, it produces a *trace* (counterexample) => useful also to identify bugs, not only to prove correctness
- SW vs. HW model checking
 - can be applied also to hardware
 - e.g. Intel adopting Model-Checking after the Pentium Bug in 1994
 - used in mission critical software systems
 - e.g. NASA after Mars Polar Lander incident in 1999

MODEL-CHECKING APPLICATIONS



- Program model checking
 - application of the model-checking techniques to software systems
 - in particular to the final implementation
 - discovering software defects

DEALING WITH THE STATE-SPACE EXPLOSION PROBLEM

- The big problem of model-checking technique is the **size of the state space** Esplosione del numero di stati
 - how to manage graph of millions of states? Is it feasible ?
- State-of-the art techniques
 - applying rules to reduce the number of states
 - using variables that can be modelled by a limited number of values
 - incremental construction of the whole graph 
 - exploring only reachable state of an execution
 - checking the truth of a correctness specification as the incremental diagram is constructed, stopping the construction se ~~is~~ a falsifying state is found
 - *symbolic* model checking 
 - working with set of states

Invece di analizzare il programma con tutti i suoi dettagli reali, creiamo una versione "semplificata". Così che il grafo si rimpicciolisce drasticamente senza perdere la logica del possibile bug.

Invece di costruire l'intero grafo e poi cercare se c'è un bug, lo costruiamo piano per piano. Non analizziamo combinazioni di variabili che non possono mai verificarsi nella realtà. Mentre il computer costruisce il grafo, controlla subito se la proprietà di correttezza è violata. Se trova un errore, si ferma immediatamente. Non c'è bisogno di sprecare memoria per costruire il resto del grafo.

Questa è la tecnica più avanzata. Invece di analizzare "lo Stato A, poi lo Stato B, poi lo Stato C" uno per uno, il Model Checker usa delle formule matematiche (spesso rappresentate da strutture chiamate Binary Decision Diagrams o BDD) che descrivono migliaia di stati contemporaneamente.

SPIN AND PROMELA



Può verificare proprietà di Safety (es. "niente deadlock") e di Liveness (es. "prima o poi il messaggio arriva") usando la logica temporale LTL.

SPIN is a widely used model-checker used in both academic research and industrial software development

Prende un modello scritto in PROMELA e analizza tutti i possibili scenari di esecuzione.

- extremely efficient
- used in modelling and designing concurrent and distributed systems in general



non è un linguaggio di programmazione, ma un linguaggio di modellazione.

PROMELA is the language that is used in Spin to write concurrent programs modelling language

Linguaggio per descrivere i processi (molto simile a pseudo-codice)

- limited number of constructs intended to be used to build models of concurrent systems

AN EXAMPLE: DEKKER IN PROMELA

```
bool    wantp = false, wantq = false;
byte    turn = 1;

proctype p() { Processo p
    do ::
        wantp = true;
        do :: !wantq -> break;
            :: else ->
                if :: (turn == 1)
                    :: (turn == 2) ->
                        wantp = false;
                        (turn == 1);
                        wantp = true
                fi
            od;
        printf("LOG: p in CS\n");
        turn = 2;
        wantp = false
    od
}

proctype q() { /* similar */ Processo q
init {
    atomic {
        run p
        run q
    }
}
```

JAVA PATH FINDER (JPF)

- JPF is a recent model-checker specialized for the verification of programs written in Java
 - developed by NASA, used for critical software
 - open-source project
 - <http://javapathfinder.sourceforge.net/>
- JPF is a special JVM ^{che esegue i programmi, in teoria, lungo tutti i} ~~executing programs theoretically along all~~ possible scenarios (execution paths), checking for property violations
 - deadlocks, uncaught exceptions, etc
 - If it finds an error, JPF reports the whole execution ^{scenario} that leads to it

JPF MODEL OF OPERATION

A differenza di SPIN (che analizza modelli in PROMELA), JPF lavora direttamente sui file .class (il bytecode Java).

Choice Generators (Generatori di Scelte): Sono i punti in cui il programma può prendere strade diverse.

- Scheduling Choices: Quale thread eseguire? JPF prova tutte le permutazioni dei thread (l'interleaving).
- Data Choices: Se una variabile può valere 1, 2 o 3, JPF proverà l'esecuzione per ciascuno di questi valori.



data/scheduling
heuristics

VM
observation

MJI

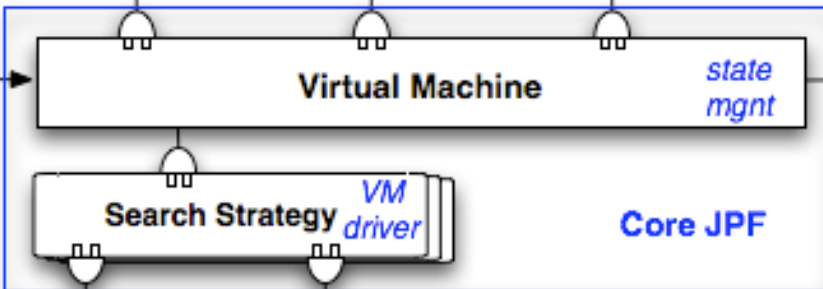
library
abstraction

choice
generator

vm
listener

verification target
(Java bytecode
program)

*.class
*.jar



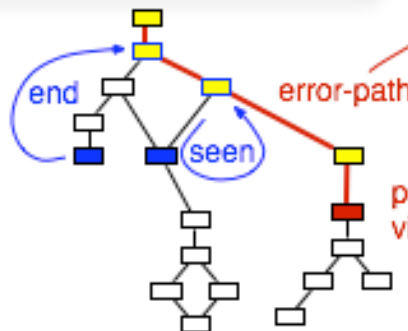
Ad ogni passo,
verifica se le proprietà
sono rispettate

property
checker

system/
apps

search
listener

search
observation



error-path

property
violation

Se trova un errore, JPF ti fornisce l'intero
scenario di esecuzione (la lista esatta di
interleaving) che ha portato a quel fallimento.

verification report



Per evitare di entrare in loop infiniti e per gestire l'esplosione degli stati, JPF memorizza ogni stato visitato.

- Ogni volta che la VM genera un nuovo stato (una combinazione di valori delle variabili, stack dei thread, heap della memoria), lo confronta con gli stati già visti.
- Se lo stato è già presente nel database ("State Matching"), JPF interrompe quel percorso e torna indietro, sapendo che quella strada è già stata analizzata.

Mentre una JVM standard (come quella di Oracle o OpenJDK) esegue una sola sequenza di istruzioni alla volta, JPF è una Virtual Machine di backtracking.

- Quando arriva a una "scelta" (ad esempio, quale thread deve muoversi ora o quale valore può assumere un input), JPF salva lo stato corrente.
- Esplora un percorso fino alla fine o fino a un errore.
- Torna indietro (backtrack) allo stato salvato e prova l'altra strada possibile.

VERIFICATION TECHNIQUES (2/2): INDUCTIVE PROOF OF INVARIANTS

Oltre al Model
Checking

- **invariant**

- a formula that must be ~~invariantly~~^{always} true at any point of any ~~computation~~^{scenario}

- e.g. $\square \neg (p_3 \wedge q_3)$

- Invariants can be ~~proved~~^{dimostrate} using **induction** over the states of all the computations:

Un invariante è una proprietà che deve rimanere vera per tutta la durata dell'esecuzione del programma, in ogni possibile stato e dopo ogni possibile azione

- to prove that A is an invariant:

Dimostri che A è vera non appena il programma viene avviato

- prove that A is true in the initial state (*the base case*)

- assume that A is true in a generic state S (*inductive hypothesis*) and prove that A is true in all the possible state next to S (*inductive step*)

Assumi che la proprietà A sia vera in un generico stato S qualsiasi. Non sai quale sia, ma ipotizzi che fino a quel momento tutto sia andato bene.

Dimostri che, se A è vera nello stato S, allora rimarrà vera in tutti i possibili stati successivi (S') raggiungibili con una singola azione atomica.

- **Deductive systems**

Perché funziona: Se è vera all'inizio (base), e se ogni azione possibile preserva la verità (passo), allora per "effetto domino" la proprietà sarà vera per sempre.

la dimostrazione automatica dei teoremi

→ software systems for ~~automated theorem proving~~

Theorem Provers: Questi sistemi non esplorano stati, ma manipolano formule logiche. Verificano che i passaggi matematici della tua dimostrazione siano corretti.

SAFETY AND LIVENESS PROPERTY VERIFICATION

- Safety property are easier to verify

- – a safety property must be true at all states
 - it is sufficient to find a state not verifying the property to complete the verification
- a liveness property claims that a state satisfying a property will inevitably occur
 - it is not sufficient to check states one by one, it is necessary to check all possible scenarios
 - > it requires more complex theory and software techniques

Verifica basata sui singoli stati: Per dimostrare che una proprietà di Safety è vera, devi assicurarti che sia rispettata in ogni singolo stato raggiungibile del programma.

È facile trovare un errore (controesempio): Se la proprietà è falsa, significa che esiste almeno uno stato nel grafo in cui il "fatto brutto" accade.

Per dimostrare che una proprietà di liveness è falsa, devi trovare un intero percorso infinito (una sequenza di transizioni) in cui la promessa non viene mai mantenuta.

TEMPORAL LOGIC OF ACTIONS (TLA/TLA+)

Oltre a JPF, sono presenti altri Model Checker

- A formal specification language introduced by Leslie Lamport
 - based on simple discrete math (set theory and predicates)
 - <http://research.microsoft.com/en-us/um/people/lamport/tla/tla.html>
- Used to specify and verify complex real-world systems
 - es: Amazon Cloud
 - "Use of Formal Methods at Amazon Web Services"
 - <http://research.microsoft.com/en-us/um/people/lamport/tla/amazon.html>

Logica per descrivere
proprietà di
correttezza e fare
Model Checking

TLA+ SPEC

Non è un linguaggio per scrivere software, ma per scrivere specifiche.

in modo matematico

- A TLA+ specification describes the set of all possible legal behaviors (execution traces) of a system.

TLA+ definisce quali stati iniziali S_0 sono validi e quali transizioni di stato ($S_i \rightarrow S_{i+1}$) sono legali (cioè permesse dalle regole del sistema).

- can be used to describe both
 - the desired correctness properties of the system (the 'what')
 - the design of the system (the 'how')
- TLA+ objective: make it possible to show that a system design correctly implements the desired correctness properties
 - either via conventional mathematical reasoning
 - by using tools such as the TLC model checker
 - a tool which takes a TLA+ specification and exhaustively checks the desired correctness properties across all of the possible execution traces
- PlusCal (formerly called +CAL)
 - an algorithm language based on TLA+
 - for writing algorithms, just as a programming language is for writing programs
 - a PlusCal algorithm is translated to a TLA+ specification, which can be checked with the TLC model checker

System Design: Definisci come i tuoi processi interagiscono (es. "Il processo A invia un messaggio, poi aspetta la conferma").

L'obiettivo: Dimostrare matematicamente che il "HOW" soddisfa sempre il "WHAT".

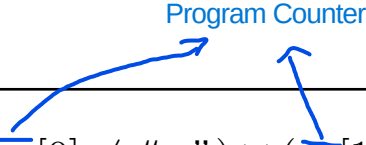
Prende la tua specifica e genera tutti gli stati possibili per verificare che le proprietà definite valgano sempre.

TLA+ può essere molto ostico perché somiglia più a una formula matematica che a del codice. Per questo è stato creato PlusCal, un linguaggio che somiglia a uno pseudocodice di programmazione (ha cicli while, variabili, istruzioni if), ma con la potenza logica di TLA+.

PLUSCAL EXAMPLE

```
--algorithm Peterson {  
  variables  $flag = [i \in \{0, 1\} \mapsto \text{FALSE}], turn = 0;$   
  process ( $proc \in \{0, 1\}$ ) {  
    a0: while (TRUE) {  
      a1:   $flag[self] := \text{TRUE};$   
      a2:   $turn := \text{Not}(self);$   
      a3a: if ( $flag[\text{Not}(self)]$ ) {goto a3b} else {goto cs} ;  
      a3b: if ( $turn = \text{Not}(self)$ ) {goto a3a} else {goto cs} ;  
      cs:  skip; \* critical section  
      a4:   $flag[self] := \text{FALSE};$   
    } \* end while  
  } \* end process  
} \* end algorithm
```

Fig. 1. Peterson's algorithm in PlusCal.


$$MutualExclusion \triangleq (\underline{pc}[0] \neq \text{"cs"}) \vee (\underline{pc}[1] \neq \text{"cs"})$$

THEOREM $Spec \Rightarrow \square MutualExclusion$

BIBLIOGRAPHY

- “Principles of Concurrent and Distributed Programming” - M. Ben-Ari. Addison-Wesley
 - chapter 1, 2, 3, 4
- Further suggested reading & references
 - “What good temporal logic is?” - Lamport, 1983
 - “Transition Systems” - M. Nygaard, 2004
 - “Principle of Model Checking” - Baier & Katoen